

Lab 6

February 2, 2026

Linked Lists

A linked list is a data structure in which the objects are arranged in a linear order. Unlike an array, where the order is determined by indices, the order in a linked list is determined by pointers stored in each object.

Each node of a linked list L has two attributes:

- `key`: stores the data value,
- `next`: a pointer to the next node in the list.

The attribute $L.\text{head}$ points to the first node of the list. If $L.\text{head} = \text{NIL}$, the list is empty. For the last node in the list, the attribute `next` is equal to NIL .

Searching a Linked List

The procedure `LIST-SEARCH` returns a pointer to the first node with key k , or NIL if no such node exists. The worst-case running time is $\Theta(n)$.

`LIST-SEARCH( $L, k$ )`

```
1   $x = L.\text{head}$ 
2  while  $x \neq \text{NIL}$  and  $x.\text{key} \neq k$ 
3       $x = x.\text{next}$ 
4  return  $x$ 
```

Prepending to a Linked List

The procedure `LIST-PREPEND` inserts a node x at the beginning of the linked list. If the list is empty (i.e., $L.\text{head} = \text{NIL}$), the new node becomes the only node in the list. Otherwise, the `next` pointer of x is set to the current head. In both cases, the head pointer is updated to point to x . This operation runs in $O(1)$ time.

`LIST-PREPEND( $L, x$ )`

```
1  if  $L.\text{head} = \text{NIL}$ 
2       $x.\text{next} = \text{NIL}$ 
3  else  $x.\text{next} = L.\text{head}$ 
4   $L.\text{head} = x$ 
```

After the operation, node x becomes the new head of the list. This operation runs in $O(1)$ time.

Inserting After a Given Node

The procedure LIST-INSERT-AFTER inserts a node x immediately after a given node y in the linked list. If $y = \text{NIL}$, the procedure performs no action. Otherwise, the insertion is performed by updating the next pointer of y and the next pointer of x . This operation runs in $O(1)$ time.

LIST-INSERT-AFTER(y, x)

```
1  if  $y = \text{NIL}$ 
2      return
3   $x.\text{next} = y.\text{next}$ 
4   $y.\text{next} = x$ 
```

Deleting from a Linked List

The procedure LIST-DELETE removes a node x from the linked list. If $x = \text{NIL}$, the procedure performs no action. Since the list is singly linked, a pointer to the node preceding x , denoted by $prev$, must also be provided. If x is the first node in the list (i.e., $prev = \text{NIL}$), the head pointer is updated. Otherwise, the next pointer of $prev$ is updated to bypass x . This operation runs in $O(1)$ time once the appropriate pointer is known.

LIST-DELETE($L, prev, x$)

```
1  if  $x = \text{NIL}$ 
2      return
3  if  $prev = \text{NIL}$ 
4       $L.\text{head} = x.\text{next}$ 
5  else  $prev.\text{next} = x.\text{next}$ 
6
```

If x is the first node, the head pointer is updated. Otherwise, the predecessor's next pointer bypasses x . Deletion by key requires a prior call to LIST-SEARCH.

C Implementation

```
1  /* Singly linked list node */
2  struct node {
3      int key;
4      struct node* next;
5  };
6
7  /* Singly linked list */
8  struct list {
9      struct node* head;
10 };
11
12 /* List-Search(L, k) */
13 struct node* list_search(struct list* L, int k)
14 {
15     struct node* x = L->head;
16     while (x != 0 && x->key != k) {
17         x = x->next;
```

```

18     }
19     return x; /* returns NULL (0) if not found */
20 }
21
22 /* List-Prepend(L, x) with the same NIL-check structure as pseudocode */
23 void list_prepend(struct list* L, struct node* x)
24 {
25     if (L->head == 0) {
26         x->next = 0;
27     } else {
28         x->next = L->head;
29     }
30     L->head = x;
31 }
32
33 /* List-Insert-After(y, x) with y == NIL check */
34 void list_insert_after(struct node* y, struct node* x)
35 {
36     if (y == 0) {
37         return;
38     }
39     x->next = y->next;
40     y->next = x;
41 }
42
43 /* List-Delete(L, prev, x) with x == NIL check and prev == NIL case */
44 void list_delete(struct list* L, struct node* prev, struct node* x)
45 {
46     if (x == 0) {
47         return;
48     }
49     if (prev == 0) {
50         L->head = x->next;
51     } else {
52         prev->next = x->next;
53     }
54 }
55
56 /* Example calls */
57 void example()
58 {
59     struct list L;
60     struct node a, b, c;
61     struct node* p;
62
63     L.head = 0;
64
65     a.key = 10; a.next = 0;
66     b.key = 20; b.next = 0;
67     c.key = 30; c.next = 0;
68
69     /* Prepend with checks (matches pseudocode) */
70     list_prepend(&L, &a);      /* 10 */
71     list_prepend(&L, &b);      /* 20 -> 10 */

```

```

72
73     /* Insert-after with y == NIL check */
74     list_insert_after(&b, &c); /* 20 -> 30 -> 10 */
75
76     /* Search example (optional) */
77     p = list_search(&L, 30); /* p points to node c, or 0 if not found */
78
79     /* Delete with x == NIL check; delete c with prev = b */
80     list_delete(&L, &b, &c); /* 20 -> 10 */
81 }

```

Task: Conversion to a Doubly Linked List

A doubly linked list stores an additional pointer in each node that refers to the previous node in the list. This allows deletion of a node in $O(1)$ time using only a pointer to the node itself.

In a doubly linked list, each node is an object with an attribute key and two pointer attributes: next and prev. Given a node x in the list, $x.next$ points to its successor and $x.prev$ points to its predecessor. The attribute $L.head$ points to the first node of the list. If $L.head = \text{NIL}$, the list is empty. For the first node, prev is NIL, and for the last node, next is NIL.

Task. Starting from the linked list implementation above:

1. Modify the node structure to include a prev pointer.
2. Update insertion procedures to maintain both next and prev.
3. Rewrite deletion so that it removes a node using only a pointer to the node.
4. Ensure correct handling of head and tail nodes.
5. All insertion and deletion operations must run in $O(1)$ time.

Rubrik (Total: 10 Marks)

Criterion	Marks
Correctly extends the node structure to support bidirectional traversal	2
Insertion logic correctly maintains list connectivity in both directions	2
Deletion logic correctly removes a node using only a pointer to the node	2
Boundary conditions (empty list, head, and tail) are handled correctly	2
All insertion and deletion operations run in $O(1)$ time	2
Total	10